

RL Control:

Reinforcement Learning - Lecture 7

Felipe Meneguzzi
`felipe.meneguzzi@pucrs.br`

PUCRS

Material adapted from:
Martha and Adam White - University of Alberta/Coursera
Sutton and Barto - Reinforcement Learning: An Introduction (2nd ed.)
David Silver - Reinforcement Learning

Outline

- 1 Recap
- 2 On-Policy Monte-Carlo Control
Exploration in Monte Carlo
- 3 On-Policy Temporal-Difference Control
- 4 Off-Policy Temporal-Difference Control
Importance Sampling
Q-Learning

- Last lecture:
 - Prediction
 - Monte-Carlo and Temporal Difference Learning
- This lecture (Sutton and Barto 5.3-5.6 and 6.4-6.6)
 - **Control**
 - On and Off-Policy Control using MC and TD
- Next lecture
 - n-Step Bootstrapping
 - Eligibility Traces

Recap

- We understand how to plan in stochastic environments for both short-term and long-term goals via:
 - MDP formalization
 - Dynamic Programming algorithms
- These formalisms assume we have both $\mathcal{R}$ and $\mathcal{P}$
- We also know how to compute the value function via RL Prediction:
 - Monte-Carlo
 - Temporal-Difference
- How can we turn predictions into optimal policies?

Uses of Model-Free Control

Some example problems that can be modeled as MDPs

- Elevator
- Parallel Parking
- Ship Steering
- Bioreactor
- Helicopter
- Aeroplane Logistics
- Robocup Soccer
- Quake
- Portfolio management
- Protein Folding
- Robot walking
- Game of Go

For most of these problems, either:

- MDP model is unknown, but experience can be sampled
- MDP model is known, but is too big to use, except by samples

Model-free control can solve these problems

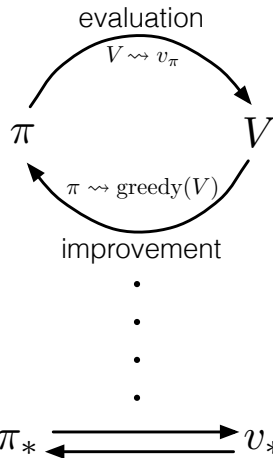
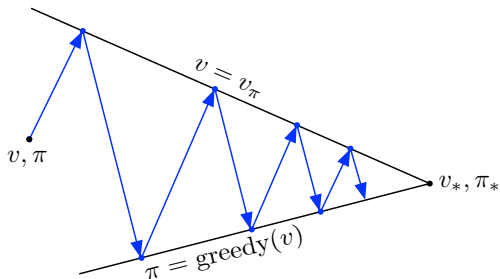
Outline

- 1 Recap
- 2 On-Policy Monte-Carlo Control
Exploration in Monte Carlo
- 3 On-Policy Temporal-Difference Control
- 4 Off-Policy Temporal-Difference Control
Importance Sampling
Q-Learning

On and Off-policy Learning

- **On-policy** learning
 - “Learn on the job”
 - Learn about policy π from experience sampled from π
- **Off-policy** learning
 - “Look over someone’s shoulder”
 - Learn about policy π from experience sampled from π_b

Generalized Policy Iteration (Refresher)



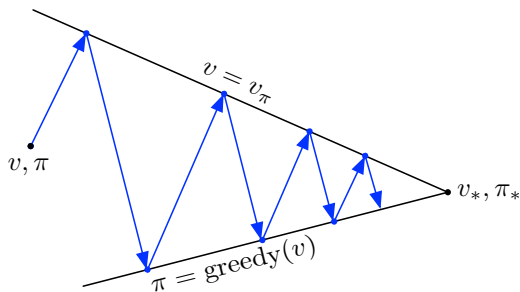
Policy evaluation Estimate v_π

Any policy evaluation

Policy improvement Generate $\pi' \geq \pi$

Any policy improvement algorithm

Generalized Policy Iteration With Monte-Carlo Evaluation



Policy evaluation Monte-Carlo policy evaluation $V = v_\pi$?

Policy improvement Greedy policy improvement?

Model-free Policy Iteration Using Action-Value Function

- Greedy policy improvement over $V(s)$ requires model of MDP

$$\pi'(s) = \arg \max_{a \in \mathcal{A}} \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a V(s')$$

- Greedy policy improvement over $Q(s, a)$ is model-free

$$\pi'(s) = \arg \max_{a \in \mathcal{A}} Q(s, a)$$

Monte-Carlo Methods in Reinforcement Learning

Recall that

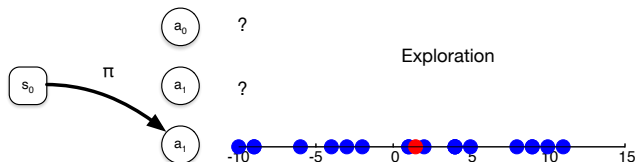
$$v_{\pi}(s) = \mathbb{E}_{\pi} [G_t \mid S_t = s]$$

Monte-Carlo Methods in Reinforcement Learning

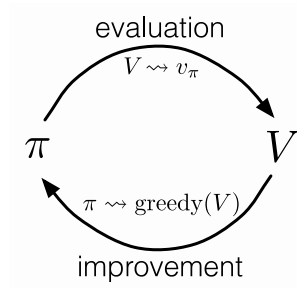
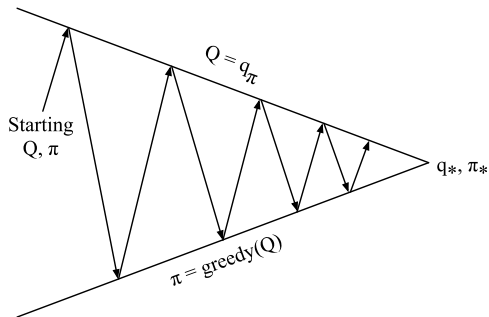
Recall that

$$q_{\pi}(s, a) = \mathbb{E}_{\pi} [G_t \mid S_t = s, A_t = a]$$

Actions-values are useful for learning a policy



Generalized Policy Iteration With Action-Value Functions



Policy evaluation Monte-Carlo policy evaluation, $Q = q_\pi$

Policy improvement Greedy policy improvement?

Example of Greedy Action Selection



"Behind one door is tenure - behind the other
is flipping burgers at McDonald's."

Copyright © 2003 David Farley, d-farley@ibiblio.org

- There are two doors in front of you
- You open the left door and get reward 0
 $v(\text{left}) = 0$
- You open the right door and get reward +1
 $v(\text{right}) = +1$
- You open the right door and get reward +3
 $v(\text{right}) = +2$
- You open the right door and get reward +2
 $v(\text{right}) = +2$
- $\vdots$
- Are you sure you've chosen the best door?

Exploring Starts

s_0, a_0
Random

S			
			G

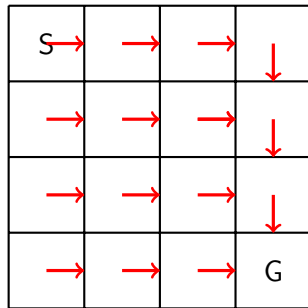
Exploring Starts

s_0, a_0 $s_1, a_1, s_2, a_2, \dots$
Random From π

S			
			G

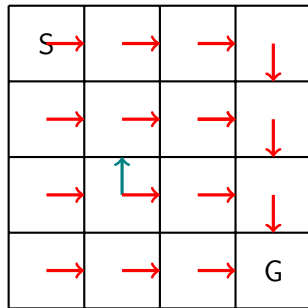
Exploring Starts

s_0, a_0 $s_1, a_1, s_2, a_2, \dots$
Random From π



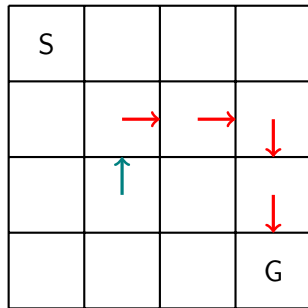
Exploring Starts

s_0, a_0 $s_1, a_1, s_2, a_2, \dots$
Random From π



Exploring Starts

s_0, a_0 $s_1, a_1, s_2, a_2, \dots$
Random From π



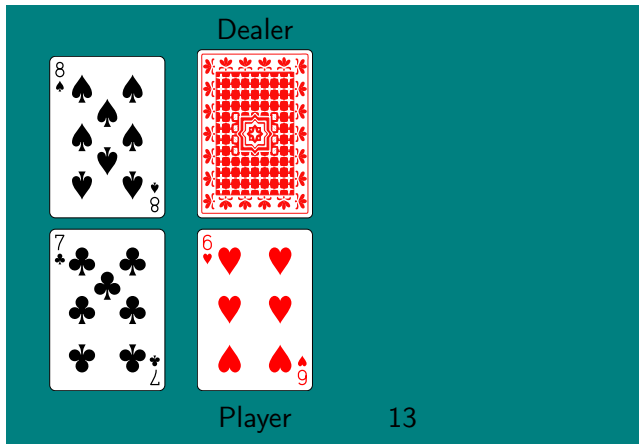
Algorithm

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

```
1: function mcESControl( )
2:   Initialize  $\pi(s) \in \mathcal{A}(s)$  (arbitrarily), for all  $s \in \mathcal{S}$ 
3:   Initialize  $Q(s, a) \in \mathbb{R}$  (arbitrarily), for all  $s \in \mathcal{S}$ 
4:    $Ret(s, a) \leftarrow$  an empty list, for all  $s \in \mathcal{S}$  ▷ This is the return
5:   loop forever (for each episode)
6:     Choose  $S_0 \in \mathcal{S}$ ,  $A_0 \in \mathcal{A}(S_0)$  randomly s.t. all pairs have probability  $> 0$ 
7:     Generate an episode from  $S_0, A_0$ , following  $\pi$ :  $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$ 
8:      $G \leftarrow 0$ 
9:     for each step of episode,  $t = T - 1, T - 2, \dots, 0$  do
10:       $G \leftarrow \gamma G + R_{t+1}$ 
11:      Append  $G$  to  $Ret(S_t)$ 
12:       $V(S_t) \leftarrow \text{average}(Ret(S_t))$ 
13:       $\pi(S_t) \leftarrow \arg \max_a Q(S_t, a)$ 
```

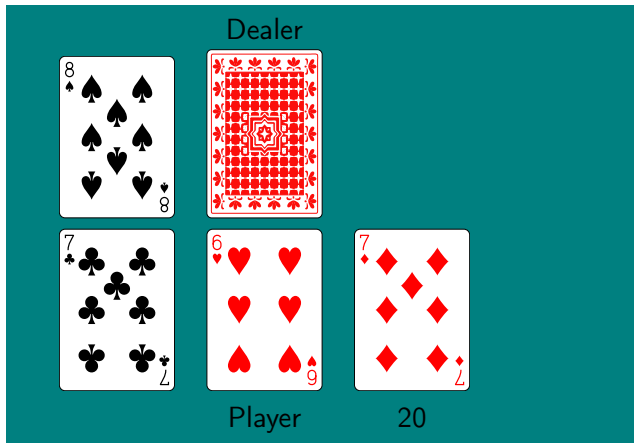
Blackjack example

S, A	$Ret(S, A)$	$Q(S)$ Hit, Stick	$\pi(S)$



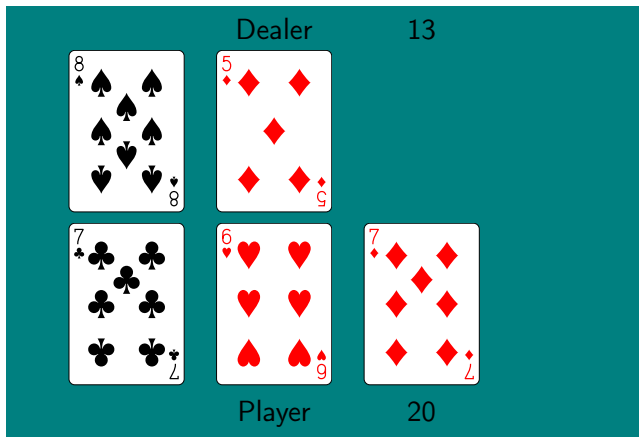
Blackjack example

S, A	$Ret(S, A)$	$Q(S)$ Hit, Stick	$\pi(S)$



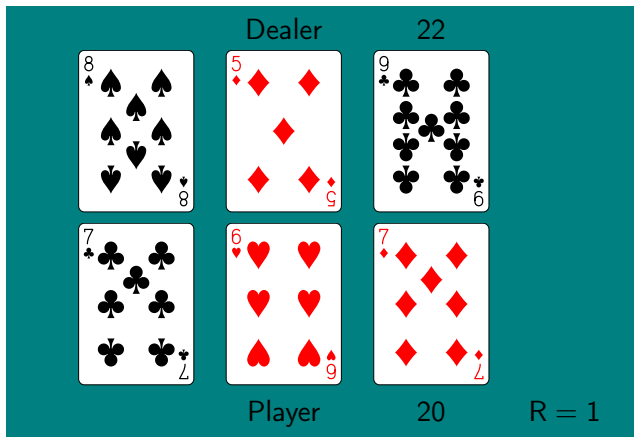
Blackjack example

S, A	$Ret(S, A)$	$Q(S)$ Hit, Stick	$\pi(S)$



Blackjack example

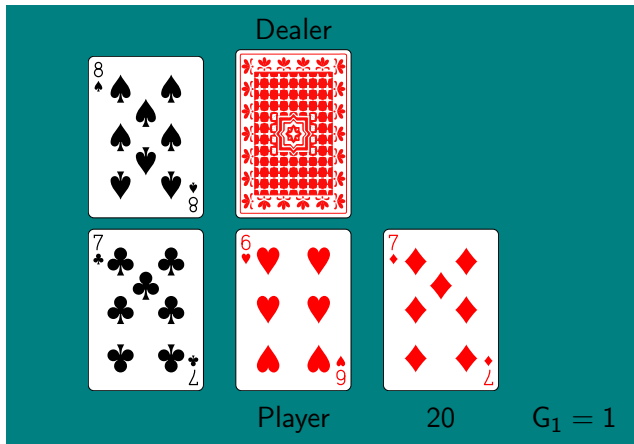
S, A	$Ret(S, A)$	$Q(S)$ Hit, Stick	$\pi(S)$



Blackjack example

S, A	$Ret(S, A)$	$Q(S)$ Hit, Stick	$\pi(S)$

$X = (\text{NoAce}, 20, 8)$

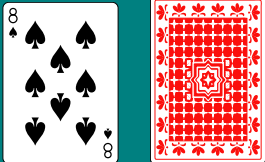


Blackjack example

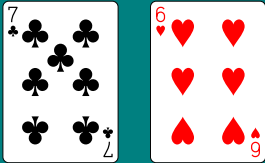
S, A	$Ret(S, A)$	$Q(S)$ Hit, Stick	$\pi(S)$
$X, Stick$	$Ret(X, Stick) = [1]$	0, 1	$Stick$

$X = (\text{NoAce}, 20, 8)$

Dealer



Player



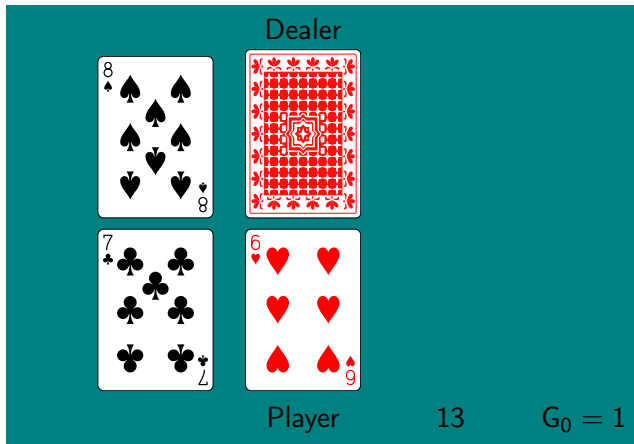
13 $G_0 = 1$

Blackjack example

S, A	$Ret(S, A)$	$Q(S)$ Hit, Stick	$\pi(S)$
$X, Stick$	$Ret(X, Stick) = [1]$	0, 1	$Stick$
Y, Hit	$Ret(Y, Hit) = [1]$	1, 0	

$X = (\text{NoAce}, 20, 8)$

$Y = (\text{NoAce}, 13, 8)$

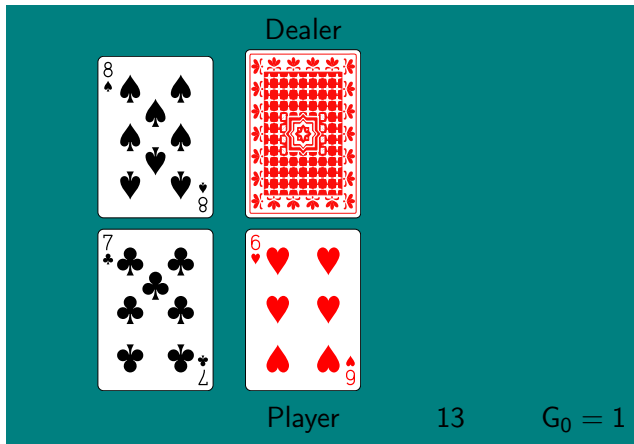


Blackjack example

S, A	$Ret(S, A)$	$Q(S)$ Hit, Stick	$\pi(S)$
$X, Stick$	$Ret(X, Stick) = [1]$	0, 1	$Stick$
Y, Hit	$Ret(Y, Hit) = [1]$	1, 0	Hit

$X = (\text{NoAce}, 20, 8)$

$Y = (\text{NoAce}, 13, 8)$



ϵ -Greedy Exploration

- Simplest idea for ensuring continual exploration
- All m actions are tried with non-zero probability
- With probability $1 - \epsilon$ choose the greedy action
- With probability ϵ choose an action at random

$$\pi(a \mid s) = \begin{cases} \frac{\epsilon}{m} + 1 - \epsilon & \text{if } a^* = \arg \max_{a \in \mathcal{A}} Q(s, a) \\ \frac{\epsilon}{m} & \text{otherwise} \end{cases}$$

ϵ -Greedy Policy Improvement

Theorem

For any ϵ -greedy policy π , the ϵ -greedy policy π' with respect to q_π is an improvement, $v_{\pi'}(s) \geq v_\pi(s)$

$$\begin{aligned} q_\pi(s, \pi'(s)) &= \sum_{a \in \mathcal{A}} \pi'(a | s) q_\pi(s, a) \\ &= \frac{\epsilon}{m} \sum_{a \in \mathcal{A}} q_\pi(s, a) + (1 - \epsilon) \max_{a \in \mathcal{A}} q_\pi(s, a) \\ &\geq \frac{\epsilon}{m} \sum_{a \in \mathcal{A}} q_\pi(s, a) + (1 - \epsilon) \frac{\pi(a | s) - \epsilon/m}{1 - \epsilon} q_\pi(s, a) \\ &= \sum_{a \in \mathcal{A}} \pi(a | s) q_\pi(s, a) = v_\pi(s) \end{aligned}$$

Therefore from policy improvement theorem $v_{\pi'}(s) \geq v_\pi(s)$

Definition

Greedy in the Limit with Infinite Exploration (GLIE)

- All state-action pairs are explored infinitely many times,

$$\lim_{k \rightarrow \infty} N_k(s, a) = \infty$$

- The policy converges on a greedy policy,

$$\lim_{k \rightarrow \infty} \pi_k(a \mid s) = 1(a = \arg \max_{a' \in \mathcal{A}} Q_k(s, a'))$$

For example, ϵ -greedy is GLIE if ϵ reduces to zero at $\epsilon_k = \frac{1}{k}$

GLIE Monte-Carlo Control

- Sample k th episode using $\pi : \{S_1, A_1, R_2, \dots S_t\} \sim \pi$
- For each state S_t and action A_t in the episode,

$$N(S_t, A_t) \leftarrow N(S_t, t) + 1$$

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)} (G_t - Q(S_t, A_t))$$

- Improve policy based on new action-value function

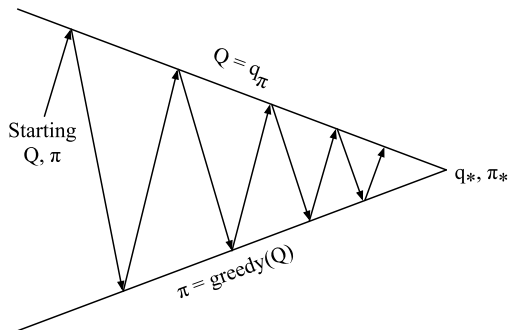
$$\epsilon \leftarrow \frac{1}{k}$$

$$\pi \leftarrow \epsilon\text{-greedy}(Q)$$

Theorem

GLIE Monte-Carlo control converges to the optimal action-value function,
 $Q(s, a) \rightarrow q_*(s, a)$

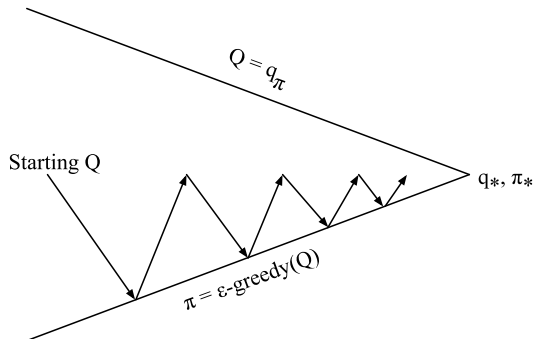
Monte-Carlo Policy Iteration



Policy evaluation Monte-Carlo policy evaluation, $Q \approx q_\pi$

Policy improvement ϵ -greedy policy improvement

Monte-Carlo Control



Every episode

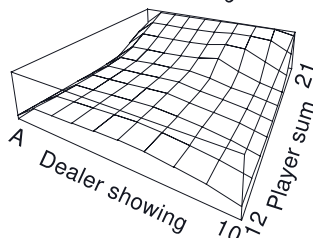
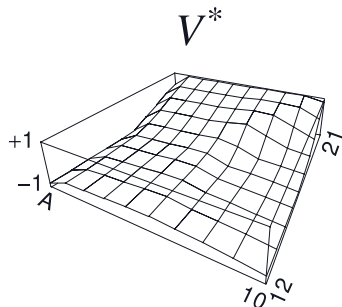
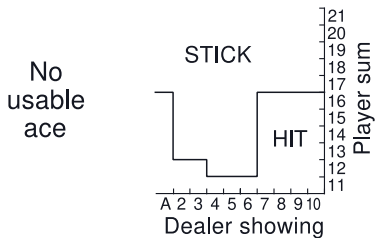
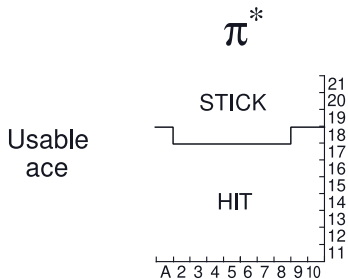
Policy evaluation Monte-Carlo policy evaluation, $Q \approx q_\pi$

Policy improvement ϵ -greedy policy improvement

Back to the Blackjack Example



Monte-Carlo Control in Blackjack



ϵ -soft policies

- ϵ -greedy policies in MC require an unrealistic assumption: exploring starts
- Alternative ϵ -soft policies: stochastic policies with non-zero probability for all actions, i.e. $\pi(a | s) > 0, \forall s \in \mathcal{S}, \forall a \in \mathcal{A}(s)$
- An ϵ -soft policy is defined as:

$$\pi(a | s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|} & \text{if } a = \arg \max_a q(s, a) \\ \frac{\epsilon}{|\mathcal{A}(s)|} & \text{if } a \neq \arg \max_a q(s, a) \end{cases}$$

- An ϵ -greedy policy is a special case of ϵ -soft
- No guarantees to reach optimal policy, but rather optimal ϵ -soft policy

ϵ -soft MC algorithm

Check Sutton and Barto Section 5.4

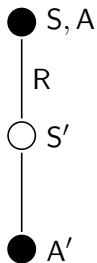
MC vs. TD Control

- Temporal-difference (TD) learning has several advantages over Monte-Carlo (MC)
 - Lower variance
 - Online
 - Incomplete sequences
- Natural idea: use TD instead of MC in our control loop
 - Apply TD to $Q(S, A)$
 - Use ϵ -greedy policy improvement
 - Update every time-step

Outline

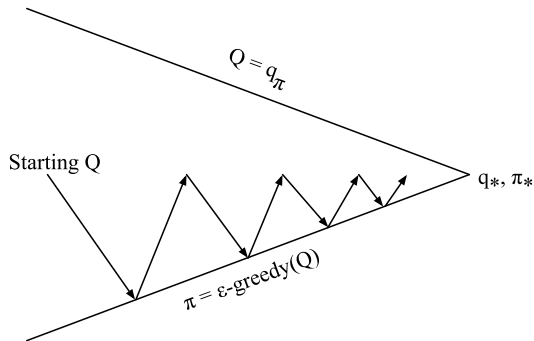
- 1 Recap
- 2 On-Policy Monte-Carlo Control
Exploration in Monte Carlo
- 3 On-Policy Temporal-Difference Control
- 4 Off-Policy Temporal-Difference Control
Importance Sampling
Q-Learning

Updating Action-Value Functions with Sarsa



$$Q(S, A) \leftarrow Q(S, A) + \alpha (R + \gamma Q(S', A') - Q(S, A))$$

On-Policy Control With Sarsa



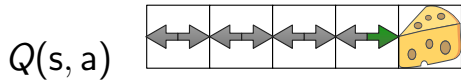
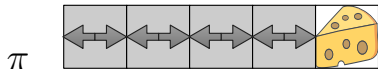
Every **time-step**

Policy evaluation **Sarsa** $Q \approx q_\pi$

Policy improvement ϵ -greedy policy improvement

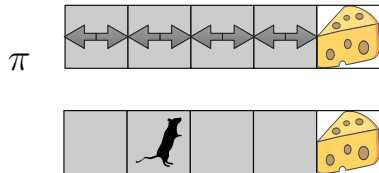
Cheese Example

Policy Evaluation



Cheese Example

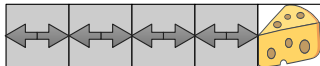
Policy Evaluation



Cheese Example

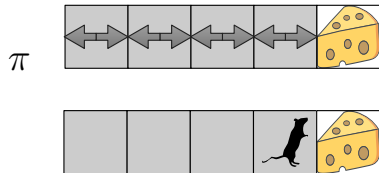
Policy Evaluation

π



Cheese Example

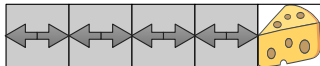
Policy Evaluation



Cheese Example

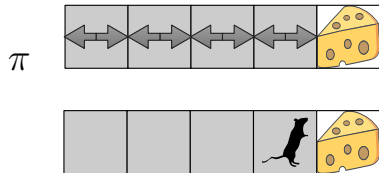
Policy Evaluation

π



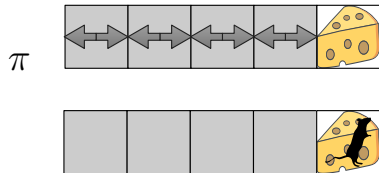
Cheese Example

Policy Evaluation



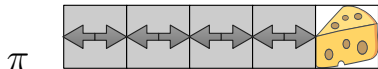
Cheese Example

Policy Evaluation

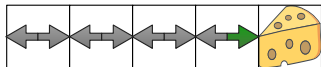


Cheese Example

Policy Evaluation

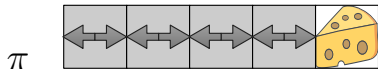


$Q(s, a)$

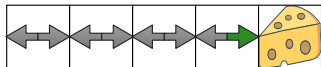


Cheese Example

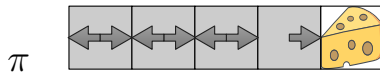
Policy Evaluation



$Q(s, a)$

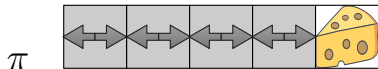


Policy Improvement

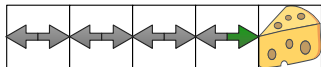


Cheese Example

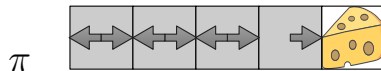
Policy Evaluation



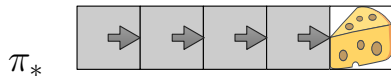
$Q(s, a)$



Policy Improvement



$\vdots$



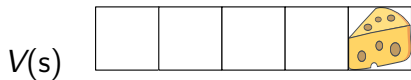
From state-values to action-values



From state-values to action-values



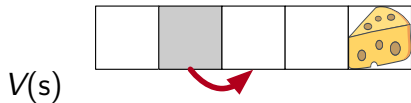
State to state



From state-values to action-values



State to state

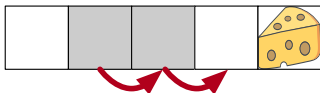


From state-values to action-values



State to state

$V(s)$

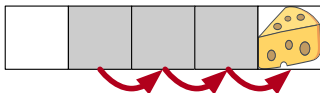


From state-values to action-values



State to state

$V(s)$

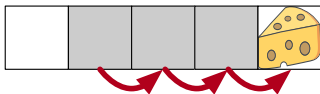


From state-values to action-values



State to state

$V(s)$

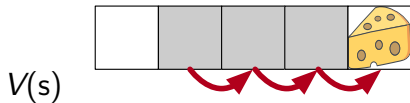


State-action to state-action

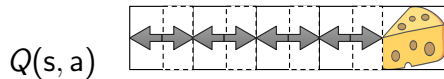
From state-values to action-values



State to state



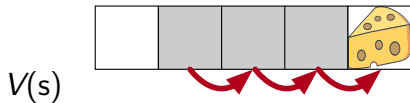
State-action to state-action



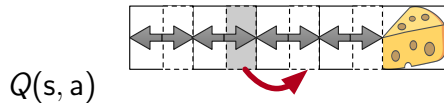
From state-values to action-values



State to state



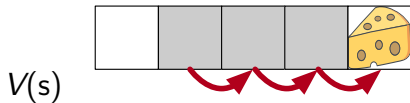
State-action to state-action



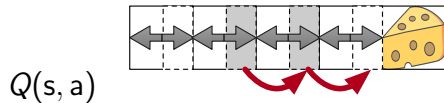
From state-values to action-values



State to state



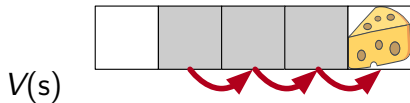
State-action to state-action



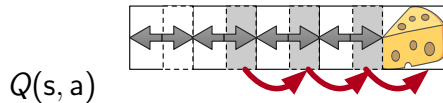
From state-values to action-values



State to state



State-action to state-action



Sarsa Algorithm for On-Policy Control

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Require: step size $\alpha \in (0, 1]$, small $\epsilon > 0$

```
1: function sarsaControl( $\alpha, \epsilon$ )
2:   Initialize  $Q(s, a) \in \mathbb{R}$ , for all  $s \in \mathcal{S}^+$ ,  $a \in \mathcal{A}(s)$  arbitrarily except that
    $Q(\text{terminal}, \cdot) = 0$ 
3:   loop for each episode
4:     Initialize  $S$ 
5:     Choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.  $\epsilon$ -greedy)
6:     loop for each step of the episode
7:       Take action  $A$ , observe  $R, S'$ 
8:       Choose  $A'$  from  $S'$  using policy derived from  $Q$  (e.g.  $\epsilon$ -greedy)
9:        $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$ 
10:       $S \leftarrow S'; A \leftarrow A'$ 
   until  $S$  is terminal
```

Convergence of Sarsa

Theorem

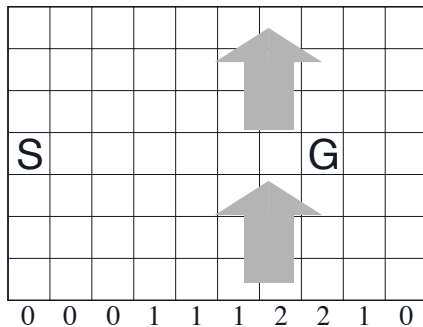
Sarsa converges to the optimal action-value function, $Q(a, s) \rightarrow q_(a, s)$, under the following conditions:*

- *GLIE sequences of policies $\pi_t(a \mid s)$*
- *Robbins-Monro sequence of step-sizes α_t*

$$\sum_{t=1}^{\infty} \alpha_t = \infty$$

$$\sum_{t=1}^{\infty} \alpha_t^2 < \infty$$

Windy Gridworld Example



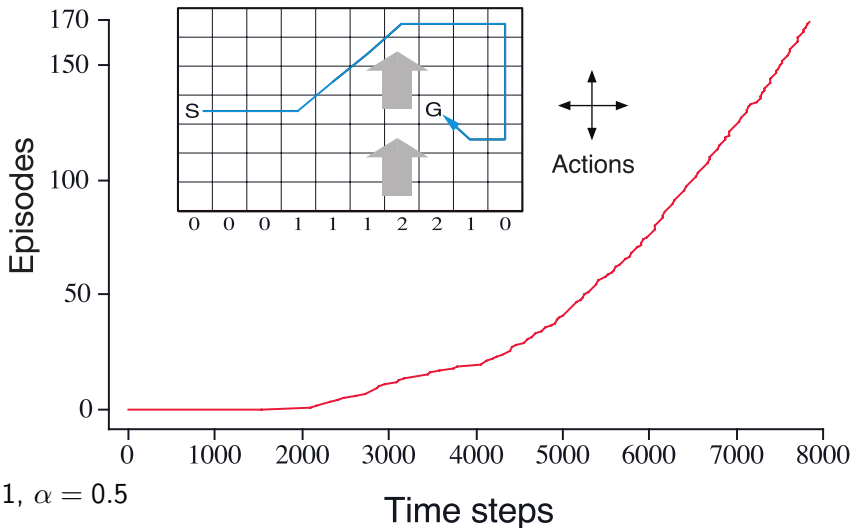
standard
moves



king's
moves

- $R = -1$ per time-step until reaching goal
- Undiscounted

Sarsa on the Windy Gridworld



Outline

- 1 Recap
- 2 On-Policy Monte-Carlo Control
Exploration in Monte Carlo
- 3 On-Policy Temporal-Difference Control
- 4 Off-Policy Temporal-Difference Control
Importance Sampling
Q-Learning

On and Off-Policy

- **On-Policy:** improve and evaluate the policy being used to select actions
- **Off-Policy:** improve and evaluate a **different** policy from the one used to select actions

Off-Policy Learning

- Evaluate target policy $\pi(a | s)$ to compute $v_\pi(s)$ or $q_\pi(s, a)$
- While following behavior policy $\pi_b(a | s)$

$$\{S_1, A_1, R_2, \dots, R_T\} \sim \pi_b$$

- Why is this important?
 - Learn from observing humans or other agents
 - Re-use experience generated from old policies $\pi_1, \pi_2, \dots \pi_{t-1}$
 - Learn about **optimal** policy while following **exploratory** policy
 - Learn about **multiple** policies while following **one** policy

Target and behavior policy

Target Policy

S			
			G

Target and behavior policy

Target Policy

$$\pi(a | s)$$

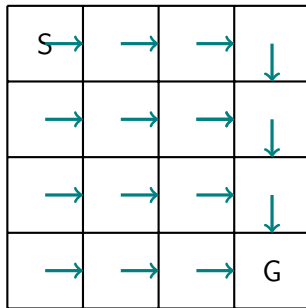
S			
			G

Target and behavior policy

Target Policy

$$\pi(a \mid s)$$

- Learn **values** for this policy
- For example, the **optimal** policy



Target and behavior policy

Behavior Policy

S			
			G

Target and behavior policy

Behavior Policy

$$\pi_b(a | s)$$

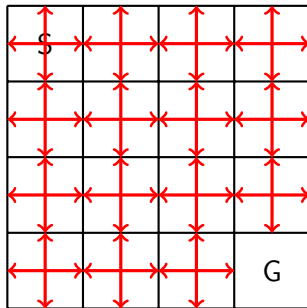
S			
			G

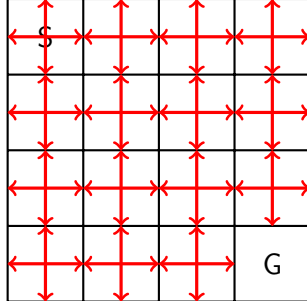
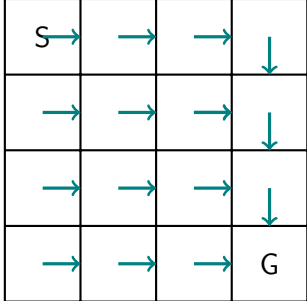
Target and behavior policy

Behavior Policy

$$\pi_b(a | s)$$

- Select **actions** from this policy
- Generally, an **exploratory** policy



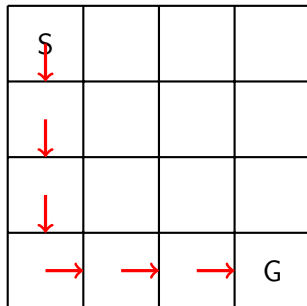


Policy Coverage

Key rule of off-policy learning: behavior policy must cover the target policy

$$\pi(a | s) > 0 \text{ where } \pi_b(a | s) > 0$$

$$\text{On policy: } \pi(a | s) = \pi_b(a | s)$$

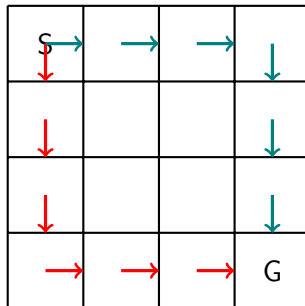


Policy Coverage

Key rule of off-policy learning: behavior policy must cover the target policy

$$\pi(a | s) > 0 \text{ where } \pi_b(a | s) > 0$$

$$\text{On policy: } \pi(a | s) = \pi_b(a | s)$$

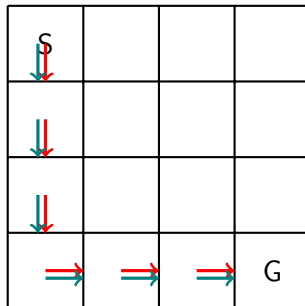


Policy Coverage

Key rule of off-policy learning: behavior policy must cover the target policy

$$\pi(a | s) > 0 \text{ where } \pi_b(a | s) > 0$$

$$\text{On policy: } \pi(a | s) = \pi_b(a | s)$$

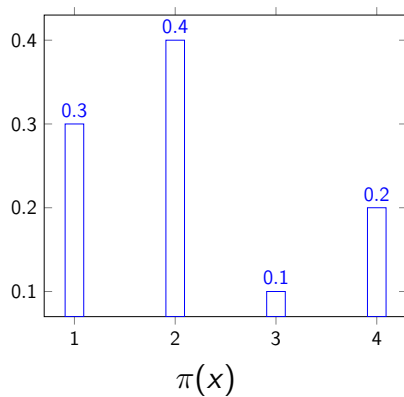
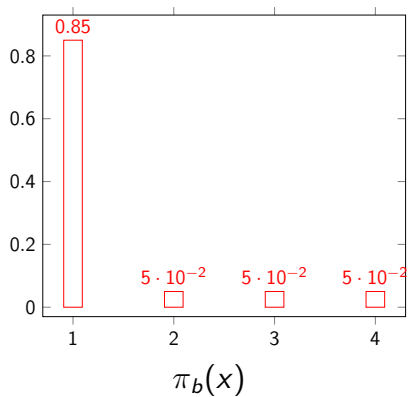


Importance Sampling

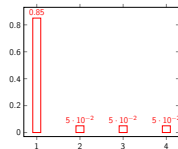
Estimate the expectation of a different distribution

$$\begin{aligned}\mathbb{E}_{X \sim P} [f(X)] &= \sum P(X) f(X) \\ &= \sum Q(X) \frac{P(X)}{Q(X)} f(X) \\ &= \mathbb{E}_{X \sim Q} \left[\frac{P(X)}{Q(X)} f(X) \right]\end{aligned}$$

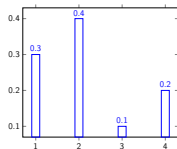
Example of importance sampling



Example of importance sampling



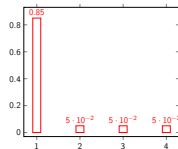
$\pi_b(x)$



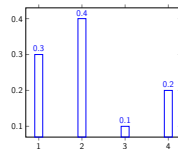
$\pi(x)$

$$\mathbb{E}_{\pi}[X] = 2.2$$

Example of importance sampling



$\pi_b(x)$



$\pi(x)$

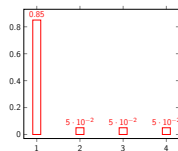
$$x = 1$$
$$\pi_b(x) = .85$$

$$x = [1]$$

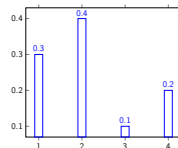
$$\pi(x) = .3$$

$$\mathbb{E}_\pi[X] = 2.2$$

Example of importance sampling



$\pi_b(x)$



$\pi(x)$

$$x = 1$$
$$\pi_b(x) = .85$$

$$x = [1]$$

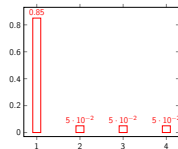
$$\pi(x) = .3$$

$$0.35$$

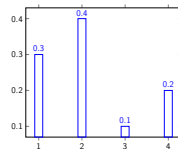
$$\mathbb{E}_\pi[X] = 2.2$$

$$\frac{1}{n} * \mathbb{E}_{x \sim \pi} [\pi_b] \rightarrow 1 * \frac{.3}{.85} = 0.35$$

Example of importance sampling



$\pi_b(x)$



$\pi(x)$

$$x = 3$$
$$\pi_b(x) = .05$$

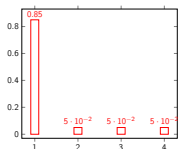
$$x = [1, 3]$$

$$\pi(x) = .1$$

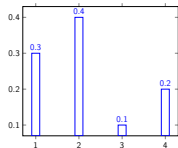
$$0.35$$

$$\mathbb{E}_\pi[X] = 2.2$$

Example of importance sampling



$\pi_b(x)$



$\pi(x)$

$$x = 3$$
$$\pi_b(x) = .05$$

$$x = [1, 3]$$

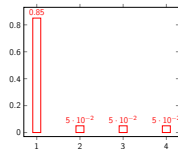
$$\pi(x) = .1$$

$$\mathbb{E}_\pi [X] = 2.2$$

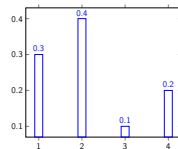
$$3.18$$

$$\frac{1}{n} * \mathbb{E}_{x \sim \pi} [\pi_b] \rightarrow \frac{(1 * \frac{.3}{.85}) + (3 * \frac{.1}{.05})}{2} = 3.18$$

Example of importance sampling



$\pi_b(x)$



$\pi(x)$

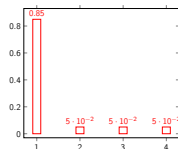
$$x = 1$$
$$\pi_b(x) = .85$$

$$x = [1, 3, 1]$$

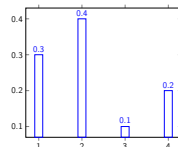
$$\pi(x) = .3$$

$$\mathbb{E}_\pi[X] = 2.2$$
$$3.18$$

Example of importance sampling



$\pi_b(x)$



$\pi(x)$

$$x = 1$$
$$\pi_b(x) = .85$$

$$x = [1, 3, 1]$$

$$\mathbb{E}_\pi[X] = 2.2$$

$$\pi(x) = .3$$

$$2.24$$

$$\frac{1}{n} * \mathbb{E}_{x \sim \pi} [\pi_b] \rightarrow \frac{(1 * \frac{.3}{.85}) + (3 * \frac{.1}{.05}) + (1 * \frac{.3}{.85})}{3} = 2.24$$

Importance Sampling for Off-Policy Monte-Carlo

- Use returns generated from π_b to evaluate π
- Weight return G_t according to similarity between policies
- Multiply importance sampling corrections along whole episode

$$G_t^{\pi/\pi_b} = \frac{\pi(A_t | S_t)\pi(A_{t+1} | S_{t+1})}{\pi_b(A_t | S_t)\pi_b(A_{t+1} | S_{t+1})} \cdots \frac{\pi(A_T | S_T)}{\pi_b(A_T | S_T)} G_t$$

- Update value towards **corrected** return

$$V(S_t) \leftarrow V(S_t) + \alpha \left(G_t^{\pi/\pi_b} - V(S_t) \right)$$

- Cannot use if π_b is zero when π is non-zero
- Importance sampling can dramatically increase variance

Importance Sampling for Off-Policy TD

- Use TD targets generated from π_b to evaluate π
- Weight TD target $R + \gamma V(S')$ by importance sampling

$$V(S_t) \leftarrow V(S_t) + \alpha \left(\frac{\pi(A_t | S_t)}{\pi_b(A_t | S_t)} (R_{t+1} + \gamma V(S_{t+1})) - V(S_t) \right)$$

- Much lower variance than Monte-Carlo importance sampling
- Policies only need to be similar over a single step

Q-Learning

- We now consider off-policy learning of action-values $Q(s, a)$
- **No** importance sampling is required
- Next action is chosen using behavior policy $A_{t+1} \sim \pi_b(\cdot \mid S_t)$
- But we consider alternative successor action $A' \sim \pi(\cdot \mid S_t)$
- And update $Q(S_t, A_t)$ towards value of alternative action

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (R_{t+1} + \gamma Q(S_{t+1}, A') - Q(S_t, A_t))$$

Off-Policy Control with Q-Learning

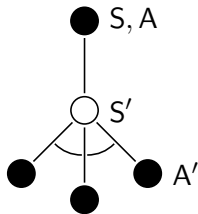
- We now allow both behavior and target policies to **improve**
- Target policy π is **greedy** w.r.t. $Q(s, a)$

$$\pi(S_{t+1}) = \arg \max_{a'} Q(S_{t+1}, a')$$

- The behavior policy π_b is e.g. ϵ -**greedy** w.r.t. $Q(s, a)$
- The Q-learning target then simplifies:

$$\begin{aligned} & R_{t+1} + \gamma Q(S_{t+1}, A') \\ &= R_{t+1} + \gamma Q(S_{t+1}, \arg \max_{a'} Q(S_{t+1}, a')) \\ &= R_{t+1} + \max_{a'} \gamma Q(S_{t+1}, a') \end{aligned}$$

Q-Learning Control Algorithm



$$Q(S, A) \leftarrow Q(S, A) + \alpha \left(R + \gamma \max_{a'} Q(S, a') - Q(S, A) \right)$$

Theorem

Q-learning control converges to the optimal action-value function, $Q(s, a) \rightarrow q_(s, a)$*

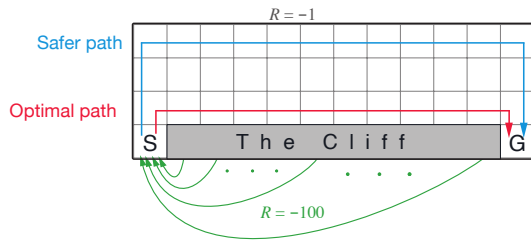
Q-Learning Algorithm for Off-Policy Control

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

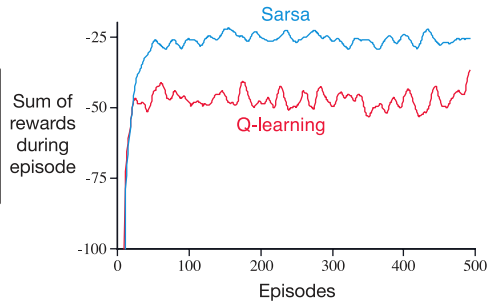
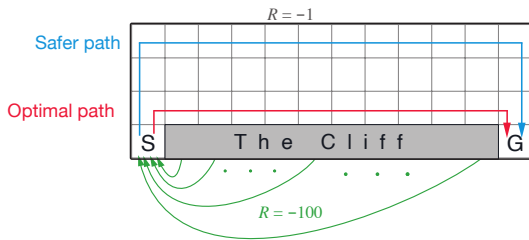
Require: step size $\alpha \in (0, 1]$, small $\epsilon > 0$

- 1: **function** tdControl(α, ϵ)
- 2: Initialize $Q(s, a) \in \mathbb{R}$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$ arbitrarily except that $Q(\text{terminal}, \cdot) = 0$
- 3: **loop** for each episode
- 4: Initialize S
- 5: **loop** for each step of the episode
- 6: Choose A from S using policy derived from Q (e.g. ϵ -greedy)
- 7: Take action A , observe R, S'
- 8: $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_{a'} Q(S', a') - Q(S, A)]$
- 9: $S \leftarrow S'$
 until S is terminal

Cliff Walking Example



Cliff Walking Example



How is Q-learning off-policy?

$$\textbf{SARSA: } Q(S, A) \leftarrow Q(S, A) + \alpha (R + \gamma Q(S', A') - Q(S, A))$$

$$\textbf{Q-Learning: } Q(S, A) \leftarrow Q(S, A) + \alpha \left(R + \gamma \max_{a'} Q(S, a') - Q(S, A) \right)$$

How is Q-learning off-policy?

$$\text{SARSA: } Q(S, A) \leftarrow Q(S, A) + \alpha \left(R + \gamma Q(S', \underbrace{A'}_{\sim \pi}) - Q(S, A) \right)$$

$$\text{Q-Learning: } Q(S, A) \leftarrow Q(S, A) + \alpha \left(R + \gamma \max_{a'} Q(S, \underbrace{a'}_{\sim \pi_* \neq \pi}) - Q(S, A) \right)$$

The Bellman Expectation equation for action-values

$$q_{\pi}(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a' | s') q_{\pi}(s', a')$$

The Bellman Expectation equation for action-values

$$q_{\pi}(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a' | s') q_{\pi}(s', a')$$

$$\text{SARSA: } Q(S, A) \leftarrow Q(S, A) + \alpha (R + \gamma Q(S', A') - Q(S, A))$$

The Bellman Expectation equation for action-values

$$q_{\pi}(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a' | s') q_{\pi}(s', a')$$

$$\text{SARSA: } Q(S, A) \leftarrow Q(S, A) + \alpha \left(R + \gamma Q(\underbrace{S'}_{\sim \mathcal{P}_{ss'}^a}, \underbrace{A'}_{\sim \pi(a'|s')}) - Q(S, A) \right)$$

The Bellman Expectation equation for action-values

$$q_{\pi}(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a' | s') q_{\pi}(s', a')$$

$$\text{SARSA: } Q(S, A) \leftarrow Q(S, A) + \alpha \left(R + \gamma \underbrace{Q(\underbrace{S'}_{\sim \mathcal{P}_{ss'}^a}, \underbrace{A'}_{\sim \pi(a'|s')})}_{\sum_{a'} \pi(a' | S') Q(S', a')} - Q(S, A) \right)$$

Expected Sarsa

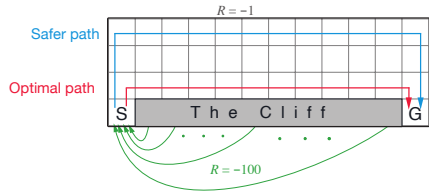
$$Q(S, A) \leftarrow Q(S, A) + \alpha \left(R + \gamma \sum_{a'} \pi(a' | S') Q(S', a') - Q(S, A) \right)$$

$$Q(S', a') = 0.0 \mid -1.0 \mid 2.0 \mid 1.0$$

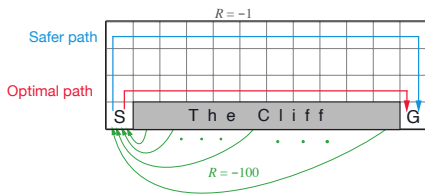
$$\pi(a' | S') = 0.1 \mid 0.1 \mid 0.7 \mid 0.1$$

$$\sum_{a'} \pi(a' | S') Q(S', a') = 0.0 - 0.1 + 1.4 + 0.1 = 1.4$$

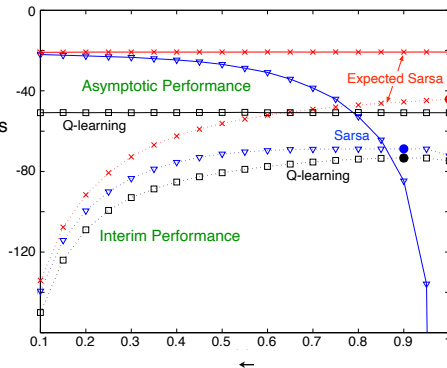
Expected Sarsa in the Cliff World



Expected Sarsa in the Cliff World

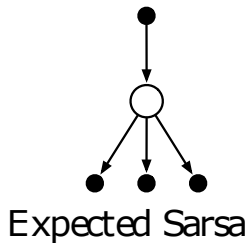
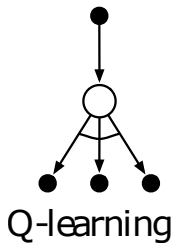


Sum of rewards
per episode



Generality of Expected Sarsa

Expected SARSA is equivalent to Q-Learning in Greedy Policy



Lecture Summary

- Control methods in Reinforcement Learning
 - Monte-Carlo Sampling
 - Temporal Difference: SARSA, Q-Learning, Expected SARSA
- On and Off-policy methods
 - Importance Sampling